



Building Java Programs
A BACK TO BASICS APPROACH 5e

Fundamentals of Computer Science

II

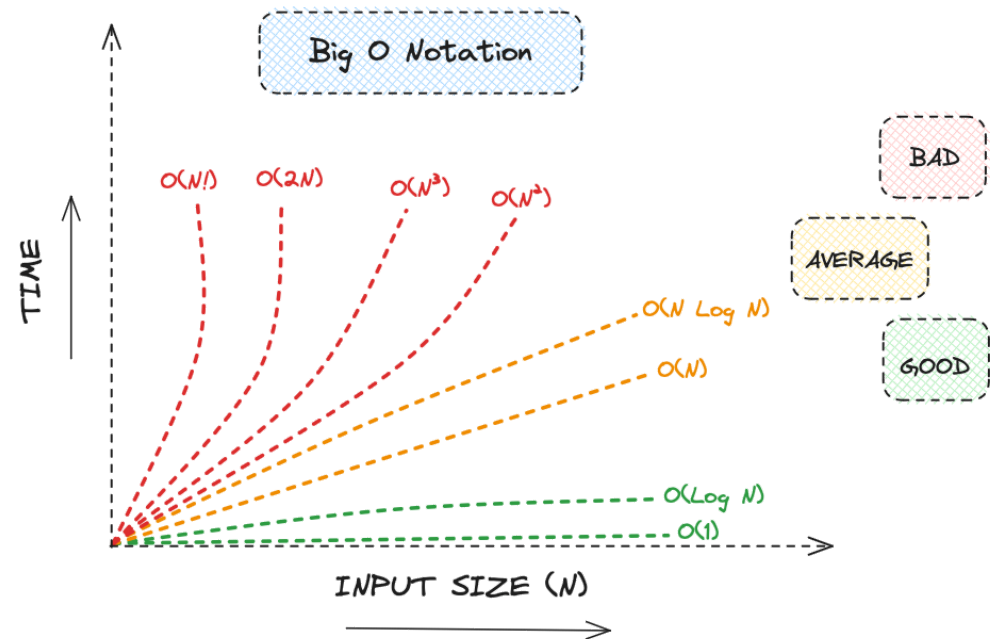
07 - Runtime

Instructor: Varik Hoang



Objectives

- Motivation
- Measuring an Algorithm's Efficiency
 - Counting Basic Operations
 - Best, Worst, and Average Cases
- Big Oh Notation
 - The Complexities
- Picturing Efficiency





Motivation

- Consider pseudocode and take a long time to **perform its computations**

```
// Algorithm A
sum = 0
for i = 1 to n
    sum = sum + i
```

```
// Algorithm B
sum = 0
for i = 1 to n
    for j = 1 to i
        sum = sum + i
```

```
// Algorithm C
sum = n * (n + 1) / 2
```

- Algorithm B takes a incredibly long time to execute
 - much longer than either of the other two algorithms
- What should we do? Use a faster computer?



Runtime Efficiency

- Time complexity: based on how long the program takes to run.
 - Empirical analysis
 - Run the program
 - Measure how long it takes
 - Algorithm analysis
 - Examine the pseudocode
 - Count the number of statements
- Space complexity: based on memory allocated for the program
 - Empirical analysis
 - Run the program
 - Measure how much memory used
 - Algorithm analysis: count the instance created



Measuring an Algorithm's Efficiency

```
// Algorithm A
sum = 0
for i = 1 to n
    sum = sum + i
```

```
// Algorithm B
sum = 0
for i = 1 to n
    for j = 1 to i
        sum = sum + i
```

```
// Algorithm C
sum = n * (n + 1) / 2
```

- How can we measure efficiency to compare various approaches?
- It would be much **better to measure an algorithm's efficiency**
- What is our concept of best?
 - Saves money, time, friend's time, or the environment?
- **Analysis of algorithms** is process of measuring the complexity of algorithms
- A **growth-rate function** is to measure how an algorithm's time grows as the problem size grows



Counting Basic Operations

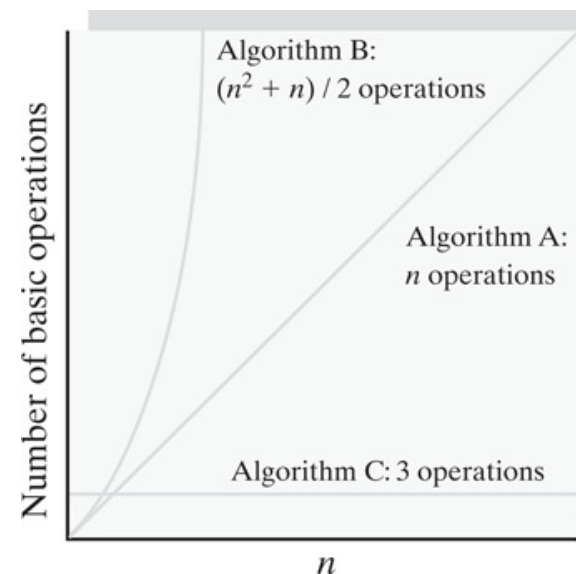
```
// Algorithm A
sum = 0
for i = 1 to n
    sum = sum + i
```

```
// Algorithm B
sum = 0
for i = 1 to n
    for j = 1 to i
        sum = sum + i
```

```
// Algorithm C
sum = n * (n + 1) / 2
```

- **Basic operations** contribute the most significantly to the algorithm's runtime
- **Ignoring operations** that are not basic to avoid final conclusion about algorithm speed
 - Variables initializations
 - Loop controls

	Algorithm A	Algorithm B	Algorithm C
Additions	n	$n(n+1)/2$	1
Multiplications			1
Divisions			1
Total basic operations	n	$(n^2 + n) / 2$	3



Common Growth-Rate Functions

- The growth-rate functions include
 - $1 < \log(\log n) < \log n < \log_2 n < n \log n < n^2 < n^3 < 2^n < n!$

n	$\log(\log n)$	$\log n$	$\log^2 n$	n	$n \log n$	n^2	n^3	2^n	$n!$
10	2	3	11	10	33	10^2	10^3	10^3	10^5
10^2	3	7	44	100	664	10^4	10^6	10^{30}	10^{94}
10^3	3	10	99	1000	9966	10^6	10^9	10^{301}	10^{1435}
10^4	4	13	177	10,000	132,877	10^8	10^{12}	10^{3010}	$10^{19,335}$
10^5	4	17	276	100,000	1,660,964	10^{10}	10^{15}	$10^{30,103}$	$10^{243,338}$
10^6	4	20	397	1,000,000	19,931,569	10^{12}	10^{18}	$10^{301,030}$	$10^{2,933,369}$



Complexity Classes

Class	Big-Oh	Double N	Example
Constant	$O(1)$	Unchanged	10ms
Logarithmic	$O(\log(N))$	Increases slightly	175ms
Linear	$O(N)$	Doubles	3.2 seconds
Log-Linear	$O(N \log(N))$	Slightly More Than Doubles	6 seconds
Quadratic	$O(N^2)$	Quadruples	1 minute 42 seconds
Cubic	$O(N^3)$	Multiplies By 8	55 minutes
...	...	...	...
Exponential	$O(2^N)$	Multiplies drastically	5×10^{61} years



Best - Worst - Average Cases

- Some algorithms have execution times that
 - Depend on **size of data set**
 - Depend on **its data itself**
- The search algorithm examines each value in array until found the desired one
 - The **best-case** is when the algorithm takes the least time
 - The **worst-case** is when the algorithm takes the most time
 - The **average-case** execution time is very useful but **hard to estimate**
- Note: The **average-case** execution time is **not the average of the best-case and worst-case** times



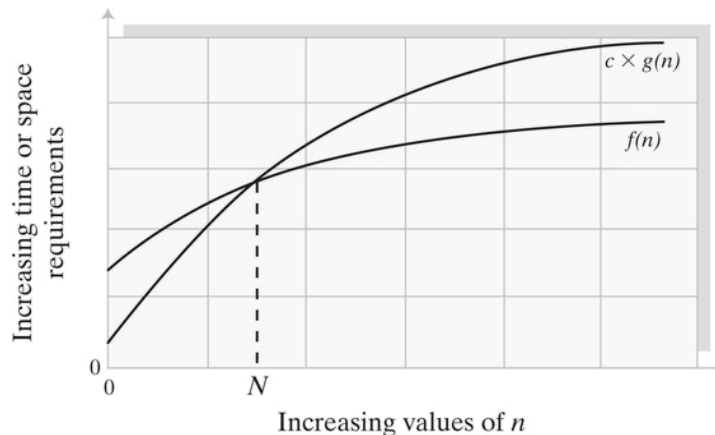
Big Oh Notation

```
// Algorithm A  
sum = 0  
for i = 1 to n  
    sum = sum + i
```

```
// Algorithm B  
sum = 0  
for i = 1 to n  
    for j = 1 to i  
        sum = sum + i
```

```
// Algorithm C  
sum = n * (n + 1) / 2
```

- We call this notation Big Oh since it uses the capital letter O
 - A has a time proportional to n , we say that A is $O(n)$
 - B has a time proportional to n^2 , we say that B is $O(n^2)$
 - C always requires three basic operations, we say that C is $O(1)$





Other Notations

- **Big Oh (O)** produces a maximum time requirement for an algorithm
 - Example: find **4** in the array [8, 2, 7, 5, 9, 6, 0, 3, 1, 4]
 - Need N comparisons to find **4** in the array
- **Big Omega (Ω)** produces a minimum time requirement for an algorithm
 - Example: find **8** in the array [8, 2, 7, 5, 9, 6, 0, 3, 1, 4]
 - Need 1 comparison to find **8** in the array
- **Big Theta (Θ)** produces a time for an algorithm encompassing both best and worst-case time complexities
 - Example: find **max** in the array [8, 2, 7, 5, 9, 6, 0, 3, 1, 4]
 - Example: find **min** in the array [8, 2, 7, 5, 9, 6, 0, 3, 1, 4]
 - Always need N comparisons to find **max/min** in the array

Picturing Efficiency

```
// Algorithm A  
sum = 0  
for i = 1 to n  
    sum = sum + i
```

```
for i = 1 to n  
    sum = sum + i
```



Picturing Efficiency (cont.)

```
// Algorithm B  
sum = 0  
for i = 1 to n  
  for j = 1 to i  
    sum = sum + 1
```

```
for i = 1 to n  
{  for j = 1 to i  
    sum = sum + 1  
}
```



Picturing Efficiency (cont.)

```
// Algorithm B  
sum = 0  
for i = 1 to n  
  for j = 1 to n  
    sum = sum + 1
```

```
for i = 1 to n  
{  for j = 1 to n  
    sum = sum + 1  
}
```





Execution Time Observations

- The time requirement for an $O(1)$ algorithm is independent of the problem size n
- What happens if we double the problem size?
 - The time requirement for an $O(\log n)$ algorithm will change, but not by much
 - An $O(n)$ algorithm will need twice the time
 - an $O(n^2)$ algorithm will need four times the time
 - an $O(n^3)$ algorithm will need eight times the time
 - Doubling the problem size for an $O(2^n)$ algorithm squares the time

Growth-Rate Function for Size n Problems	Growth-Rate Function for Size $2n$ Problems	Effect on Time Requirement
1	1	None
$\log n$	$1 + \log n$	Negligible
n	$2n$	Doubles
$n \log n$	$2n \log n + 2n$	Doubles and then adds $2n$
n^2	$(2n)^2$	Quadruples
n^3	$(2n)^3$	Multiplies by 8
2^n	2^{2n}	Squares



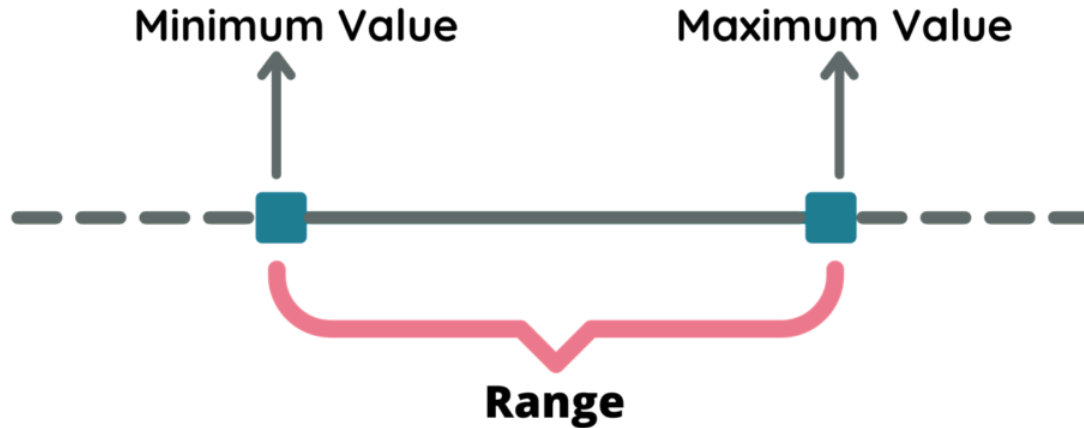
Execution Time Observations (cont.)

- How long will it take an algorithm to solve a problem whose size is one million?
 - We **cannot answer** this question exactly without knowing the algorithm
- We can use $O(n^2)$, $O(n^3)$, or $O(2^n)$ algorithms as long as problem size is small
 - $O(n^2)$ algorithm takes 1s to solve a problem size **1000**
 - $O(n^3)$ algorithm takes 1s to solve a problem size is **100**
 - $O(2^n)$ algorithm takes 1s to solve a problem size is **20**

Growth-Rate Function g	$g(10^6) / 10^6$
$\log n$	0.0000199 seconds
n	1 second
$n \log n$	19.9 seconds
n^2	11.6 days
n^3	31,709.8 years
2^n	$10^{301,016}$ years

Runtime Efficiency - Case Study

- We are asked to create a method that takes an array of integers
- The method return the **range** of the array
- The **range** is the difference between the max and min

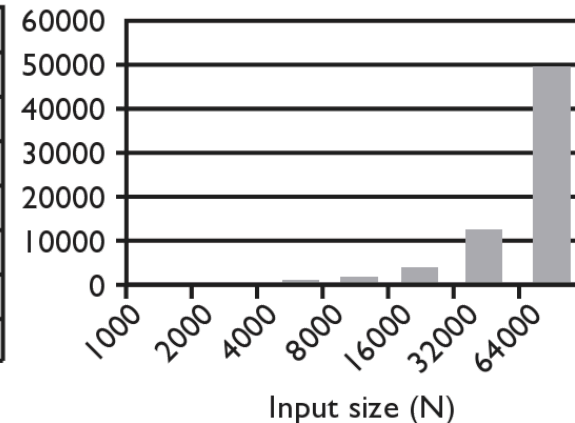


Runtime Efficiency - Case Study - Algorithm #1

```
11 class RangeVersion1
12 {
13     // returns the range of values in the given array;
14     // the difference between elements furthest apart
15     // example: range({17, 29, 11, 4, 20, 8}) is 25
16     public static int range(int[] numbers)
17     {
18         int maxDiff = 0;
19         // look at each pair of values
20         for (int i = 0; i < numbers.length; i++)
21         {
22             for (int j = 0; j < numbers.length; j++)
23             {
24                 int diff = Math.abs(numbers[j] - numbers[i]);
25                 if (diff > maxDiff)
26                 {
27                     maxDiff = diff;
28                 }
29             }
30         }
31         return maxDiff;
32     }
33 }
```

Time Complexity: $O(N^2)$

N	Runtime (ms)
1000	15
2000	47
4000	203
8000	781
16000	3110
32000	12563
64000	49937

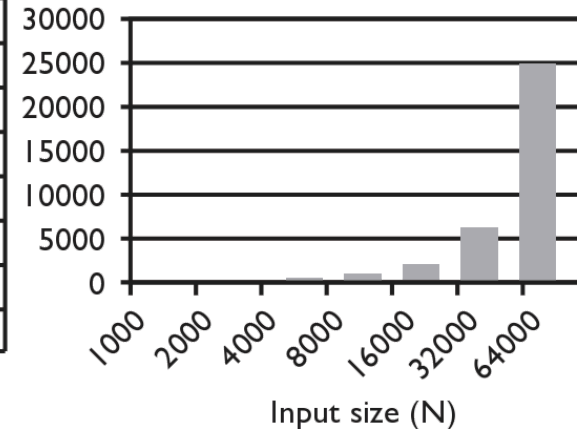


Runtime Efficiency - Case Study - Algorithm #2

```
35 class RangeVersion2
36 {
37     // returns the range of values in the given array;
38     // the difference between elements furthest apart
39     // example: range({17, 29, 11, 4, 20, 8}) is 25
40     public static int range(int[] numbers)
41     {
42         int maxDiff = 0;
43         // look at each pair of values
44         for (int i = 0; i < numbers.length; i++)
45         {
46             for (int j = i + 1; j < numbers.length; j++)
47             {
48                 int diff = Math.abs(numbers[j] - numbers[i]);
49                 if (diff > maxDiff)
50                 {
51                     maxDiff = diff;
52                 }
53             }
54         }
55         return maxDiff;
56     }
57 }
```

Time Complexity: $O(N^2)$

N	Runtime (ms)
1000	16
2000	16
4000	110
8000	406
16000	1578
32000	6265
64000	25031





Runtime Efficiency - Case Study - Algorithm #3

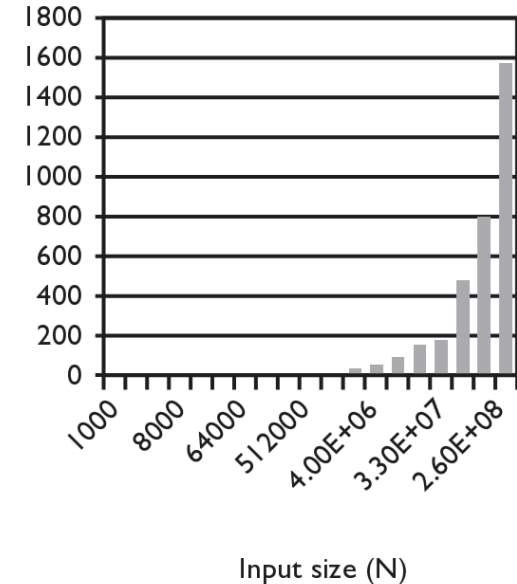
```

59 class RangeVersion3
60 {
61     // returns the range of values in the given array;
62     // the difference between elements furthest apart
63     // example: range({17, 29, 11, 4, 20, 8}) is 25
64     public static int range(int[] numbers)
65     {
66         int max = numbers[0];
67         int min = max;
68         for (int i = 1; i < numbers.length; i++)
69         {
70             if (numbers[i] < min)
71             {
72                 min = numbers[i];
73             }
74             if (numbers[i] > max)
75             {
76                 max = numbers[i];
77             }
78         }
79         return max - min;
80     }
81 }

```

Time Complexity: $O(N)$

N	Runtime (ms)
1000	0
2000	0
4000	0
8000	0
16000	0
32000	0
64000	0
128000	0
256000	0
512000	0
1e6	0
2e6	16
4e6	31
8e6	47
1.67e7	94
3.3e7	188
6.5e7	453
1.3e8	797
2.6e8	1578





Runtime Efficiency - Example #1

statement1;
statement2;
statement3; } 3

for (int i = 1; i <= N; i++) {
 statement4;
} } N

for (int i = 1; i <= N; i++) {
 statement5;
 statement6;
 statement7;
} } 3N

4N + 3

- Time Complexity: $O(N)$
- Space Complexity: $O(1)$



Runtime Efficiency - Example #2

```
for (int i = 1; i <= N; i++) {  
    for (int j = 1; j <= N; j++) {  
        statement1;  
    }  
}  
  
for (int i = 1; i <= N; i++) {  
    statement2;  
    statement3;  
    statement4;  
    statement5;  
}
```

Diagram illustrating the time complexity analysis of the code blocks:

- The first block (nested loops) is annotated with a brace indicating a complexity of N^2 .
- The second block (single loop with four statements) is annotated with a brace indicating a complexity of $4N$.
- A large brace on the right groups both blocks, indicating a total complexity of $N^2 + 4N$.

— Time Complexity: $O(N^2)$

— Space Complexity: $O(1)$



Runtime Efficiency - Try It #1

```
public static int mystery1(int n)
{
    int count = 0;
    for (int i = 0; i < n; i++)
    {
        count++;
        if (i % 2 == 0)
        {
            count++;
        }
    }
    return count;
}
```

- Time Complexity: $O(n)$
- Space Complexity: $O(1)$



Runtime Efficiency - Try It #2

```
public static int mystery2(int n)
{
    int count = 0;
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            count++;
        }
    }
    return count;
}
```

- Time Complexity: $O(n^2)$
- Space Complexity: $O(1)$



Runtime Efficiency - Try It #3

```
public static int mystery3(int n)
{
    int count = 0;
    int[] array = new int[n];
    for (int i = 0; i < 100; i++)
    {
        count++;
        count++;
    }
    return count;
}
```

- Time Complexity: $O(1)$
- Space Complexity: $O(n)$



Runtime Efficiency - Try It #4

```
public static int mystery4(int n)
{
    int count = 0;
    for (int i = 1; i < n; i = i * 2)
    {
        count++;
    }
    return count;
}
```

- Time Complexity: $O(\log(n))$
- Space Complexity: $O(1)$



Runtime Efficiency - Try It #5

```
private static String mystery5(int n)
{
    if (n == 1)
        return "1";
    else
        return mystery5(n - 1) + ", " + n;
}
```

- Time Complexity: $O(n)$
- Space Complexity: $O(n)$



The Efficiency of An Array-Based Implementation

— What are the best case, worst case, and average case of each method below?

```
public boolean add(Type newEntry)
{
    boolean success = false;
    if (!isArrayFull())
    {
        entries[size] = newEntry;
        size++;
        success = true;
    }
    return success;
}
```

```
public boolean isArrayFull()
{
    return size >= entries.length;
}

public Type[] toArray()
{
    @SuppressWarnings("unchecked")
    Type[] result = (Type[]) new Object[size];
    for (int index = 0; index < size; index++)
    {
        result[index] = entries[index];
    }
    return result;
}
```

- The **isArrayFull()** method has $\Theta(1)$
- The **add(Type newEntry)** method has $\Theta(1)$
- The **toArray** method has $\Theta(n)$



The Efficiency of An Linked Implementation

— What are the best case, worst case, and average case of each method below?

```
public boolean add(Type newEntry)
{
    // add to beginning of chain
    Node newNode = new Node(newEntry);
    newNode.next = first;
    first = newNode;
    size++;
    return true;
}
```

```
private int getIndexOf(Type anEntry)
{
    int index = 0;
    Node current = first;
    while (current != null)
    {
        if (current.data.equals(anEntry))
            return index;

        current = current.next;
        index++;
    }
    return -1;
}
```

- The **add**(Type newEntry) method has $\Theta(1)$
- The **getIndexOf** method has $O(n)$, $\Omega(1)$, and average case is n

Questions & Answer





Thank You!

A black and silver fountain pen is positioned diagonally below the end of the 'Thank You!' text. The pen has a black barrel with silver-colored accents and a silver-colored nib. It is resting on a white surface, creating a soft reflection.